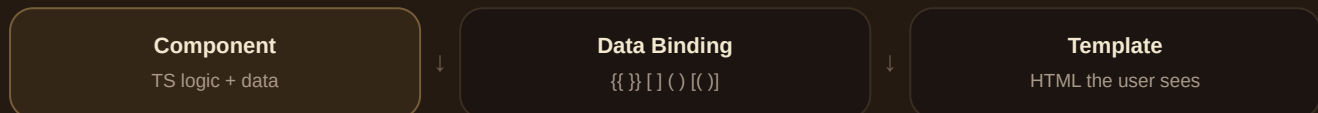


Data Binding

Angular needs a bridge between your TypeScript class and the HTML the user actually sees — and a way to react when the user interacts back. This guide walks through all four kinds, when to reach for each one, and a full worked example that uses them together.

1 · THE BIG PICTURE

How Angular connects the pieces



Pipes format the data on the way out, and the Router decides which component + template pair is shown for the current URL — binding is the wire that runs between all of it.

WITHOUT BINDING	WITH BINDING
You'd write manual <code>document.querySelector</code> + DOM updates for every value	Angular re-renders the HTML for you whenever component data changes
Every date, price, label formatted by hand, inconsistently	Pipes give every value the same look across the app

2 · THE PROBLEM BINDING SOLVES

We have data inside a component — how do we show it?

TS	HTML — how?
<pre>name = 'Ahmed';</pre>	<pre><h2>???</pre>

Angular needs a bridge between your TypeScript class and the HTML the user actually sees — and a way to react when the user interacts back. That bridge is one of four binding syntaxes, each with its own direction of data flow.

3 · INTERPOLATION

{{ }} — displaying data

<pre>name = 'Ahmed'; <h2>Hello {{ name }}</h2> <!-- Result: Hello Ahmed --></pre>
--

Use `{{ }}` whenever you just need to show a value — text, a number, or the result of a simple expression. It only ever reads a value; it never lets the user change it.

Real-world usage: user names & greetings, product titles & descriptions, counters and totals, status messages.

4 · PROPERTY BINDING

[] — binding to element properties

Problem: what if an attribute's value needs to come from a variable — like an image URL?

Static

```

```

Dynamic — []

```
imageUrl = 'assets/avatar.png';  
<img [src]="imageUrl">
```

The [] tells Angular "read this as a component *property*, not plain text." Works on any DOM property — `src`, `disabled`, `class`, and more.

Real-world usage: product & avatar images, disabling a button while loading, toggling a CSS class, setting placeholder text.

5 · EVENT BINDING

() — listening to user actions

Problem: how does Angular know the user clicked a button?

```
<button (click)="login()">Login</button>
```

```
login() { this.status = 'in'; }
```

User clicks

(click)



Component method

runs



UI update

re-renders

The () means "listen for this event, then run this method." It works for any DOM event: `click`, `input`, `submit`, `keyup`, and more.

Real-world usage: login buttons, add-to-cart, form submit, menu toggles.

6 · TWO-WAY BINDING

[()] — keeping data and UI in sync

Problem: an input field needs to update a variable, and show that variable's current value — at the same time.

```
username = '';
```

```
<input [(ngModel)]="username">  
<!-- requires importing FormsModule -->
```

Template

[()]



Component

username

Two-way binding is really just [] and () combined — property binding in, event binding out, wired to the same name.

Real-world usage: search boxes, settings forms, live filters, profile editing.

7 · CHOOSING THE RIGHT BINDING

Rule of thumb

1

Data flows out

Use `{{ }}` or `[ ]`

2

Data flows in

Something must flow back from the user — use `( )`

3

Both at once

Use `[( )]`

8 · REAL-WORLD USAGE

Data binding in a real Angular screen

Every production Angular screen — profile pages, product cards, login forms, dashboards — is built from this same small set of tools:

WHAT'S ON SCREEN	BINDING USED
Avatar image	<code>[src]</code> — property binding
Name & status text	<code>{{ }}</code> — interpolation
"Change Status" button	<code>()</code> — event binding

9 · EXERCISE SOLUTION

A user profile component, using every tool

```
userName = 'Sara';
imageUrl = 'assets/sara.png';
status = 'Available';
toggleStatus() { this.status = 'Busy'; }

<h2>{{ userName }}</h2>
<img [src]="imageUrl">
<p>{{ status }}</p>
<button (click)="toggleStatus()">Change Status</button>
```

- `userName` → `{{ }}` — just needs to be shown as text
- `imageUrl` → `[src]` — sets the `img` element's property
- `status` → `{{ }}` — displayed as read-only text
- `Change Status` → `(click)` — reacts to the user's click